1.1 Introduction to package fsimp

The `fsimp` package provides a 'fullsimp' simplification function for expressions. This code was adapted from David Scherfgen's dssimp.mac code posted on the Maxima discussion list.

This code implements a simplification routine for Maxima CAS expressions using the core simplification routines for expressions invovling trigonometric, logarithmic, exponential, polynomial and other functions.

To install the package with mxpm, run `mxpm install --path` on a copy of the repository. Without mxpm, copy the repository into a directory under `~/.maxima`, which Maxima 5.47 and later search, or add its directory to `file_search_maxima` and `file_search_lisp`, or load it by its full path. Then

```
load("fullsimp");
```

loads the package. `load("fsimp")` also works. To simplify an expression `expr` call

```
fullsimp(expr);
```

The functions, variables and rules the package defines all start with `fs`, apart from `fullsimp` itself, so loading it leaves the names a user works with alone. An input with `gamma` or a factorial loads Maxima's `gamma_simp` share package, which defines functions of its own in the session; excluding its routines, as in `fullsimp(e, gamma_simp, factorial_simp)`, keeps it from loading.

Loading the package also undoes one change that Maxima itself makes. Maxima's `trigsimp`, which `fullsimp` uses, declares `a` as a pattern variable with `matchdeclare` when it first loads, so that `a` matches any expression in rules the user later defines with `defrule`, `tellsimp` or `let`. Loading `fsimp.mac` loads `trigsimp` at once and removes that declaration, unless `a` already had one. The rules of `trigsimp` are compiled by then, so `trigsimp` works as before. A user who wants `a` as a pattern variable can declare it again with `matchdeclare`.

The algorithm begins by searching through an expression for e.g. trig, log, exp, etc. and choosing a set of core simplification routines. Then it runs a poor-man's kind of simulated annealing routine. It's poor-man's because it's not really simulated annealing, but while it cycles through various core simplification routines (several times) it allows the intermediate expression to become slightly larger if it ultimately becomes smaller. The output is always the smallest size expression, where the size is computed by the lisp `conssize` function plus the string length of the expression, with each variable, and each name of a function Maxima has no simplifier for (such as an undefined `f`), counted as one character. Renaming variables or such functions therefore leaves the size of every expression unchanged.

E. Majzoub.

1.2 Functions and Variables for fsimp

`fsdebug` [Variable]

Setting the variable `fsdebug` to 1 will allow the user to see the fsimp function operations, list selection of simplification routines, etc.

`fullsimp` (`expr`,[*simp1*,*simp2*,...]) [Function]

The `fullsimp` function takes an input expression `expr` and attempts to find a simpler form using the builtin simplification routines in core Maxima. These include: `[resimplify, expand, combine, radcan, ratsimp, rootscontract, xthru,`

multthru, factor, sqrtdenest, triglist,exptlist, loglist]. Additional trigonometric simplifications are applied as well. The full list of simplifications is shown in the variable simplist which will be shown if the variable fsdebug is set to 1.

There are two ways to manipulate the simplifications in simplist. First, the option arguments to fullsimp, simp1, simp2, etc. are simplification routines the user wants *excluded* from simplist.

Second, the user may create a *list* called fs_custom_simp, and place any user defined simplification routines. For example the code

```
fs_custom_simp:[lambda([q], block([algebraic : true], ratsimp(q)))]
```

will create a simplification routine where the variable algebraic is set to true, while the ratsimp simplification is applied.

In general, fullsimp tries to find the smallest equivalent expression based on the Common Lisp conssize and string size of an expression, counting each variable and each undefined function name as one character.

The user may manipulate the order of simplifications through use of the exclusion option and the custom simplification option.

fullsimp is a simplifying function and can be added to the command line as shown in the last example below.

```
(%i1) load("fsimp.mac")$
(%i2) fullsimp((1+cos(t))/sin(t));
                                        t
(%o2)                                cot(-)
                                        2
(%i3) fullsimp(cos(x)+%i*sin(x));
                                      %i x
(%o3)                                 %e
(%i4) (cos(t)+1)/((%i*sin(p)-cos(p))*sin(t)),fullsimp;
                                      %i p     t
(%o4)                          - %e     cot(-)
                                            2
```

If you wish to exclude particular simplification routines, add them to the arguments of fullsimp as shown below.

```
(%i1) load("fsimp.mac")$
(%i2) fullsimp((1+cos(t))/sin(t),fstrigsimp,trigsimp,trigreduce);
                                    cos(t) + 1
(%o2)                               ----------
                                      sin(t)
```

Appendix A Function and Variable index

`fullsimp` 1

`fsdebug` 1